Faculty of Computing

BSc (Hons) in Software Engineering
Year 2 Semester 2 (2026)

SE2020 - Web and Mobile Technologies Lab 10

Advanced React Lab – User Preferences App

Step 01: Create React Project

1. Open terminal and run:

   ```
   npm create vite@latest user-preferences-app
   ```

2. Choose:

   - Framework: **React**
   - Variant: **JavaScript**

3. Navigate to project folder:

   ```
   cd user-preferences-app
   npm install
   npm run dev
   ```

4. Open the URL shown in terminal (usually http://localhost:5173/)

5. Verify that the Vite starter page is displayed

Step 02: Start coding

1. Update src/App.jsx file

   ```
   function App() {
     return <h2>User Preferences App</h2>
   }

   export default App;
   ```

2. Update src/main.jsx file

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Step 03: Create Preferences Context

1. Create file src/context/PreferencesContext.jsx

```
import { createContext, useState } from "react";

export const PreferencesContext = createContext();

export const PreferencesProvider = ({ children }) => {
  const [theme, setTheme] = useState("light");
  const [username, setUsername] = useState("Student");

  return (
    <PreferencesContext.Provider
      value={{ theme, setTheme, username, setUsername }}
    >
      {children}
    </PreferencesContext.Provider>
  );
};
```

Step 04: Wrap App with Provider

1. Update src/main.jsx file:

```
import React from 'react'
import ReactDOM from "react-dom/client";
import { createRoot } from 'react-dom/client'
import { PreferencesProvider } from './context/PreferencesContext.jsx'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <PreferencesProvider>
      <App />
    </PreferencesProvider>
  </React.StrictMode>
)
```

Step 05: Create Header Component

1. Create src/components/Header.jsx:

```jsx
import { useContext } from "react";
import { PreferencesContext } from "../context/PreferencesContext";

function Header() {
  const { theme, username } = useContext(PreferencesContext);

  return (
    <div>
      <h3>Welcome, {username}</h3>
      <p>Current Theme: {theme}</p>
    </div>
  );
}

export default Header;
```

Step 06: Create Settings Component

1. Update src/components/Settings.jsx:

```jsx
import { useContext } from "react";
import { PreferencesContext } from "../context/PreferencesContext";

function Settings() {
  const { setTheme, setUsername } = useContext(PreferencesContext);

  return (
    <div>
      <input
        placeholder="Enter username"
        onChange={(e) => setUsername(e.target.value)}
      />

      <button onClick={() => setTheme("light")}>Light</button>
      <button onClick={() => setTheme("dark")}>Dark</button>
    </div>
  );
}

export default Settings;
```

Step 07: Connect Context to UI

1. Update App.jsx:

```jsx
import { useContext } from 'react'
import { PreferencesContext } from './context/PreferencesContext'
import Header from './components/Header'
import Settings from './components/Settings'
import ItemList from './components/ItemList'

function App() {
  const { theme } = useContext(PreferencesContext);

  const appStyle = {
    backgroundColor: theme === "dark" ? "#222" : "#ffffff",
    color: theme === "dark" ? "#ffffff" : "#000000",
    minHeight: "100vh",
    padding: "20px",
  };

  return(
    <div style={appStyle}>
    <h2>User Preferences App</h2>
    <Header/>
    <Settings/>
    <ItemList/>
    </div>
  )

}

export default App;
```

Step 08: Improving UI Performance with useTransition and useDeferredValue

- When React updates a large list, the UI may freeze, lag while typing and feel slow to respond.
- React Concurrent Rendering allows to:
  - Prioritize important updates (like typing)
  - Delay expensive updates (like filtering a large list)
- UseTransition: Allows React to mark certain state updates as **low priority**, keeping the UI responsive during expensive updates
- UseDeferredValue: Creates a **delayed version of a state value**, allowing React to update expensive UI parts later.

1. Create Item List Component
   - Create a large list of items
   - Add a search input
   - Use useTransition to keep typing smooth

- Use useDeferredValue to delay filtering

Create file: src/components/ItemList.jsx

```jsx
import { useState, useMemo, useTransition, useDeferredValue } from "react";

function ItemList() {
  const [query, setQuery] = useState("");
  const [isPending, startTransition] = useTransition();

  // Create a large list of 1000 items
  const items = useMemo(() => Array.from({ length: 1000 }, (_, i) => `Item ${i+1}`), []);

  // Create a deferred version of the search text
  const deferredQuery = useDeferredValue(query);

  // Filter items using deferred value
  const filteredItems = items.filter((item) =>
    item.toLowerCase().includes(deferredQuery.toLowerCase())
  );

  return (
    <div>
      <input
        placeholder="Search items..."
        value={query}
        onChange={(e) =>
          startTransition(() => setQuery(e.target.value))
        }
      />
      <ul>
        {filteredItems.map((item) => (
          <li key={item}>{item}</li>
        ))}
      </ul>
      {isPending && <p>Updating list...</p>}
    </div>
  );
}

export default ItemList;
```

Update App.jsx

```jsx
import { useContext } from 'react'
import { PreferencesContext } from './context/PreferencesContext'
import Header from './components/Header'
import Settings from './components/Settings'
import ItemList from './components/ItemList'

function App() {
  const { theme } = useContext(PreferencesContext);

  const appStyle = {
    backgroundColor: theme === "dark" ? "#222" : "#ffffff",
    color: theme === "dark" ? "#ffffff" : "#000000",
    minHeight: "100vh",
    padding: "20px",
  };

  return(
    <div style={appStyle}>
    <h2>User Preferences App</h2>
    <Header/>
    <Settings/>
    <ItemList/>
    </div>
  )

}

export default App;
```

Step 09: Lazy Loading Components with React Suspense

- Large applications take longer to load and load unnecessary components immediately.
- Solution is to load components **only when needed**.
- React.lazy(): Loads a component **only when it is rendered**
- Suspense: Displays a **fallback UI** while the component loads
- Used commonly for Settings pages, Dashboards and Admin panels

Update App.jsx

```jsx
App.jsx
src > App.jsx > ...
import { useContext, Suspense } from 'react'
import { PreferencesContext } from './context/PreferencesContext'
import Header from './components/Header'
import ItemList from './components/ItemList'

const Settings = React.lazy(() => import("./components/Settings"));

function App() {
  const { theme } = useContext(PreferencesContext);

  const appStyle = {
    backgroundColor: theme === "dark" ? "#222" : "#ffffff",
    color: theme === "dark" ? "#ffffff" : "#000000",
    minHeight: "100vh",
    padding: "20px",
  };

  return(
    <div style={appStyle}>
    <h2>User Preferences App</h2>
    <Header/>
    <Suspense fallback={<p>Loading settings...</p>}>
      <Settings />
    </Suspense>
    <ItemList/>
    </div>
  )

}

export default App;
```